KOFvision

KOFvision is a standalone Python viewer for KOF survey files with:

- 2D and 3D visualization
- KOF polyline parsing (`09 91` ... `09 99` / `09 96`)
- surface triangulation (Delaunay)
- cut/fill volume calculation between two surfaces
- interactive section drawing and profile charts

The main app is implemented in `kof_viewer.py`.

Features

- Open one or more `.kof` files
- Plot points/polylines in 2D plan view
- Plot points/polylines in 3D with optional triangulated surface
- Toggle labels, markers, lines, triangulation, and equal aspect mode
- Load a georeferenced orthophoto/raster background in the 2D tab
- Toggle file layers and raster layer from the Layers panel
- Zoom 2D view to selected (visible) layers
- Select top and bottom surfaces and compute:
  - net volume
  - cut volume
  - fill volume
  - overlap area
  - volume source modes:
    - `KOF - KOF`
    - `Top KOF - Raster`
    - `Raster - Bottom KOF`
    - `Raster - Top KOF`
- Optional intermediate layer for split-volume calculation:
  - KOF layer
  - raster elevation layer
  - virtual constant Z plane
  - split output:
    - Top - Intermediate
    - Intermediate - Bottom
    - Total (sum of both legs)
- Load multiple raster layers, reorder them (up/down), toggle visibility, and set per-layer alpha
- Set an active single-band elevation raster used for raster-based volume modes and section reference profile
- Optionally clip raster loading to current KOF bounds with user-defined buffer
- Draw one or more section lines in 2D and open section windows with:
  - top and bottom profiles
  - optional raster reference profile (when raster volume mode is active)

    - shaded cut/fill regions
    - net/cut/fill cross-section area
- Save and load polygon masks as JSON
- Save and load section lines as JSON
- Export a section-report package:
    - profile images at fixed spacing along a baseline (for example every 1 m)
    - plan-view image with baseline and profile numbers
    - HTML report and JSON manifest
- Export loaded KOF source data to GeoPackage (points + original polylines)

Requirements

- Python 3.10+
- `numpy`
- `matplotlib`
- `scipy`
- `tkinter` (usually included with standard Python installers)

Install Python dependencies:

```
pip install numpy matplotlib scipy
```

Optional (required only for GeoPackage export from the viewer):

```
pip install geopandas shapely
```

Development/testing dependencies:

```
pip install -r requirements-dev.txt
```

Run

Start empty and load files from the UI:

```
python kof_viewer.py
```

Or preload one or more KOF files:

```
python kof_viewer.py "12_A (1).kof" "12_A(1) (1).kof"
```

Tests

Run automated regression tests:

```
python -m pytest -q
```

Package for End Users (Windows)

Build a distributable app with PyInstaller:

```
python -m pip install -r requirements-build.txt
.\scripts\build_pyinstaller.ps1
```

This produces a release folder and a shareable ZIP in `dist/`.

For full packaging and distribution details, see `docs/pyinstaller-windows.md`.

Workflow

1. Open two surface files (for example existing top and design/bottom).
2. Select Top and Bottom in the second toolbar row.
3. Choose volume Method (`raster` or `triangles`).
4. Optionally choose Intermediate (`None`, `KOF layer`, `Raster layer`, or `Constant Z`).
5. If intermediate is set, volume is reported as two legs:
    - Top - Intermediate
    - Intermediate - Bottom
6. Optionally enable Triangulation.
7. Click Compute volume.
8. Read net/cut/fill (or split results) in the toolbar summary.
9. Click Draw section and pick two points in 2D to open a section profile window.

Volume Calculation Methods

KOFvision computes signed difference as:

- `difference = top_surface - bottom_surface`
- Positive difference contributes to `cut`
- Negative difference contributes to `fill`
- `net = cut - fill` (equivalently, signed integral of `difference`)

Quick Method Comparison

Method	Typical speed	Accuracy profile	Supported inputs	Intermediate split support
`raster`	Faster (default workflow)	Grid-approximation; accuracy depends on sampling resolution (`target_cells`)	`KOF - KOF`, `Top KOF - Raster`, `Raster - Bottom KOF`, `Raster - Top KOF`	Yes (`KOF`, `Raster`, `Constant Z`)
`triangles`	Slower	Exact polygon-overlap integration on triangulated KOF surfaces	`KOF - KOF` only	No

1) `raster` method (default)

Used by `compute_volume(..., method="raster")`.

How it works:

1. Finds XY overlap of top and bottom surface bounds.
2. Builds a regular sampling grid over overlap (resolution from `target_cells`, default ~200 across max span).
3. Interpolates top and bottom elevations at cell centers.
4. Optionally applies polygon mask (only samples inside mask are counted).
5. Integrates with:
    - `net = sum(diff) * cell_area`
    - `cut = sum(diff where diff>0) * cell_area`
    - `fill = -sum(diff where diff<0) * cell_area`

Supports:

- KOF vs KOF
- KOF vs raster elevation surface
- raster elevation surface vs KOF
- split-volume with intermediate layer (`KOF`, `Raster`, `Constant Z`)

2) `triangles` method

Used by `compute_volume(..., method="triangles")`.

How it works:

1. Triangulates both KOF surfaces (Delaunay + boundary filtering).
2. Finds overlapping top/bottom triangle pairs by bounding-box prefilter.
3. Computes exact convex polygon intersections for candidate pairs.
4. Integrates linear surface difference analytically over each overlap polygon.
5. Splits positive and negative contributions into cut/fill.

This method is more exact for triangulated KOF pairs, and usually slower than `raster`.

Supports:

- KOF vs KOF only

Current limitations:

- No intermediate split-volume with `triangles`.
- No raster surface participation in `triangles`.

Intermediate Split-Volume

When `Intermediate` is set and method is `raster`, KOFvision computes two legs:

- `upper = Top - Intermediate`
- `lower = Intermediate - Bottom`

The UI reports both leg nets and total net. Intermediate must be different from top and bottom.

Boundary Tightness

Boundary tightness filters long triangulation edges (relative to median edge length) before interpolation/integration.

- Lower values: tighter boundary, more outer triangles removed.
- Higher values: looser boundary, more outer triangles retained.

Changing boundary tightness marks previous volume results as stale; recompute volume after adjustment.

Mask And Section Persistence

- Draw mask: create polygon by left-clicking vertices in 2D, close with right-click or double-click.
- Save mask...: write the current closed mask polygon to JSON.
- Load mask...: restore a previously saved mask polygon from JSON.
- Draw section: create section lines in 2D.
- Save sections...: store all currently drawn section lines as JSON.
- Load sections...: restore section lines from JSON.

Section Report Export

Use Export section report... in the second toolbar row.

Current behavior:

1. Uses the latest drawn section line as report baseline.
2. Prompts for:
    - profile spacing in meters (for example 1.0)
    - cross-section length in meters
    - number of interpolation samples per profile
    - output folder
3. Creates one timestamped output folder containing:
    - `plan_view.png`
    - `profiles/P0001.png`, `profiles/P0002.png`, ...
    - `report.html`

    - `report_manifest.json`

The HTML report includes plan view, profile summary table, and all profile images with profile IDs and station distances.

View Controls

- Zoom to selected layers: fit the 2D view to visible layers.
- Reset view: clear manual zoom and return to auto extents.

Raster Background (Orthophoto)

The 2D tab can display a raster image as background (for example orthophoto) behind KOF points and polylines.

Use the top toolbar:

- Open raster...: load raster image (`.jpg`, `.jpeg`, `.png`, `.tif`, `.tiff`, `.bmp`)
- Set elevation raster: mark one loaded single-band DEM raster for raster-based volume modes
- Raster up / Raster down: reorder raster draw order
- Remove raster: remove the active raster layer
- Raster alpha: control transparency

Additional behavior:

- Multiple rasters can be loaded simultaneously.
- Elevation-based volume modes require a supported single-band elevation raster.
- Non-elevation rasters can still be shown in 2D as visual background.

Raster visibility is controlled in the left Layers panel, where raster appears as its own layer entry.

Georeferencing Requirements

Raster placement uses a world file next to the image. Supported world-file naming:

- `.jgw`, `.jpgw`, `.jpegw` for JPEG
- `.pgw`, `.pngw` for PNG
- `.tfw`, `.tifw`, `.tiffw` for TIFF
- `.bpw`, `.bmpw` for BMP
- `.wld` fallback

Notes:

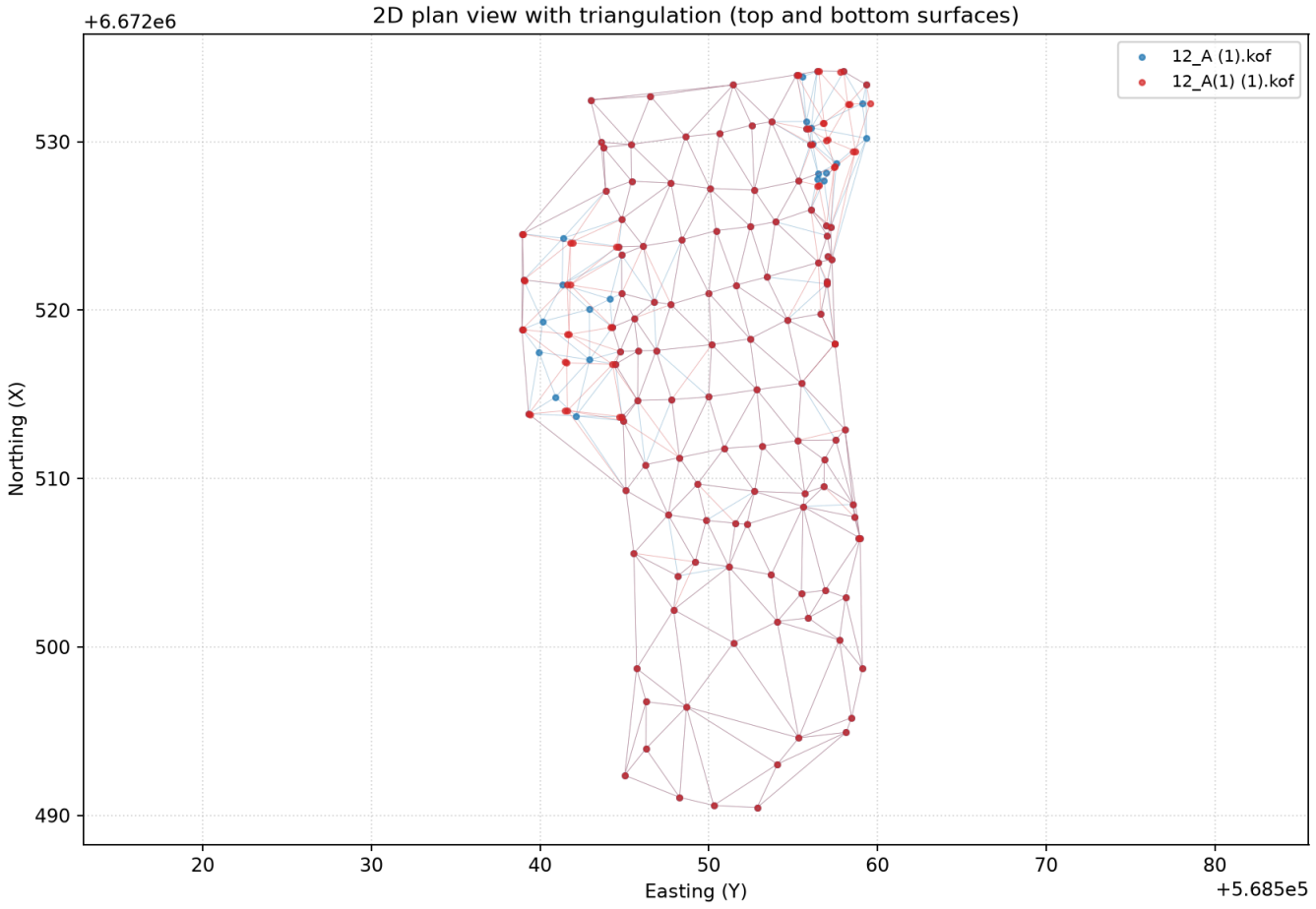
- Raster and KOF data must be in the same projected CRS for proper alignment.
- Rotated world files (non-zero rotation terms) are currently not supported.
- Very large rasters are automatically loaded as a downsampled preview for faster display; georeferencing still uses full-resolution world-file metadata.

Example Images

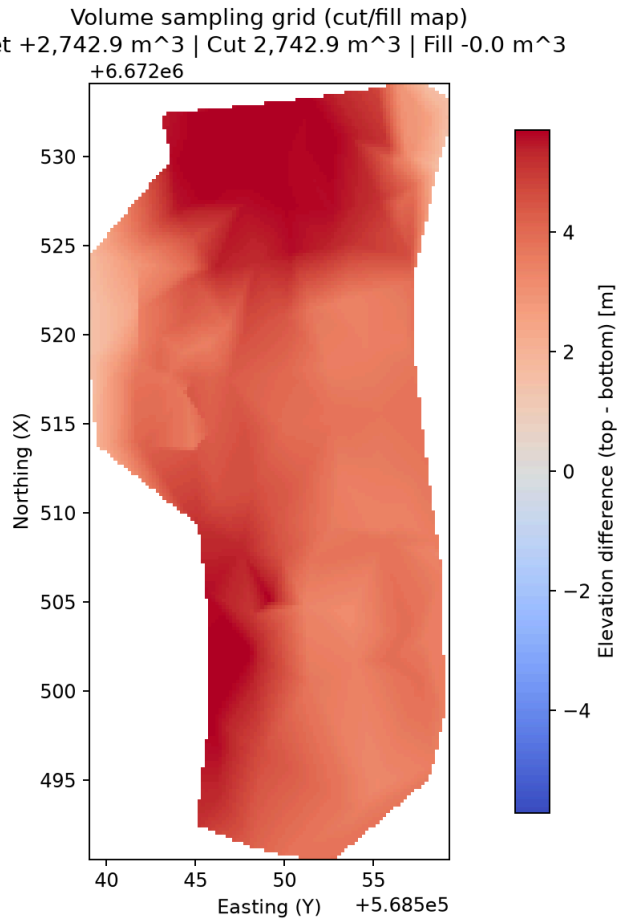
These images are generated from the included sample files:

- `12_A (1).kof`
- `12_A(1) (1).kof`

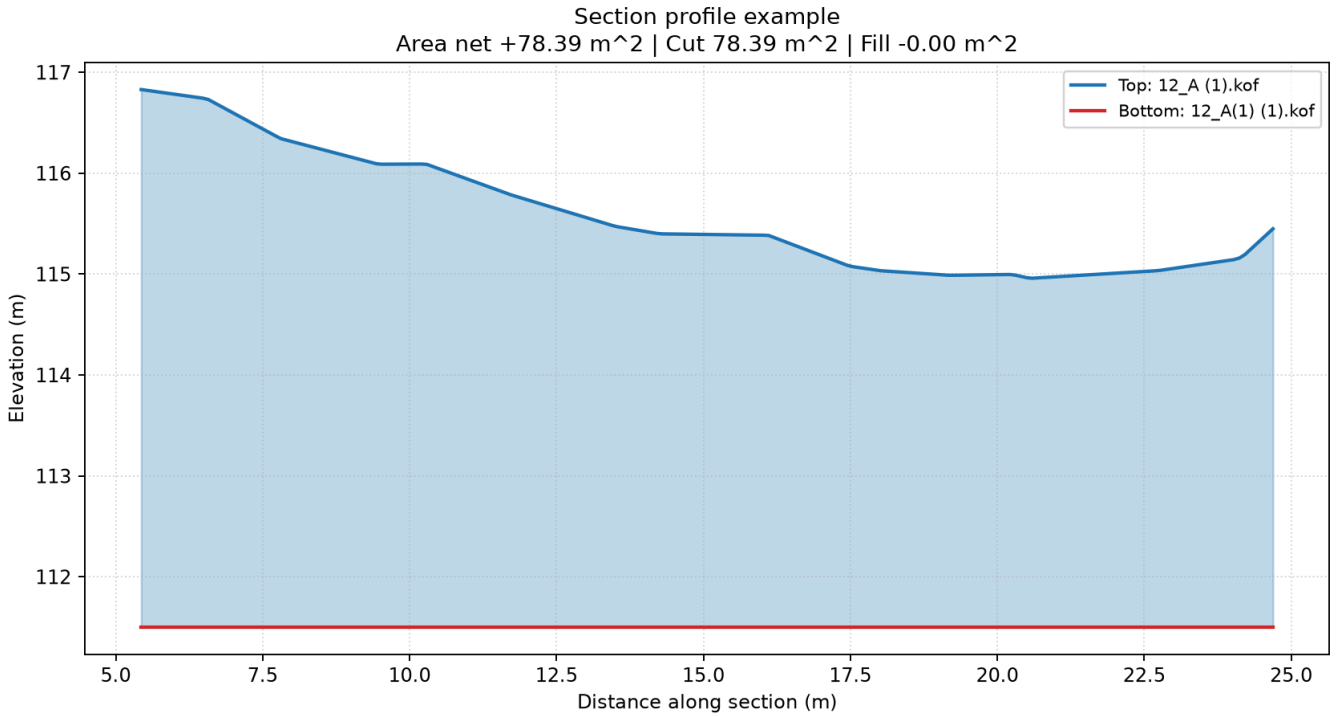
2D Plan + Triangulation



Volume Sampling (Cut/Fill Map)



Section Profile Example



Regenerate README Images

A helper script is included to regenerate the images:

```
python scripts/generate_readme_images.py
```

It writes PNG files to `docs/images/`.

Notes

- Coordinates are interpreted as:
  - `X` = Northing
  - `Y` = Easting
  - `H` = Elevation
- Volume method options:
  - `raster` (default): grid sampling over overlap between selected surfaces (KOF and/or elevation raster).
  - `triangles`: direct polygon-overlap integration on triangulated KOF pairs.
  - split-volume with intermediate layer is available only with `raster`.
- Interpolation outside the filtered triangulation boundary is ignored.